

15745 Project Proposal

Ruihang Lai and Lauren Sands

October 25, 2024

1 Project Title

Eager-Mode DNN Program Compilation with MagPy in TVM

2 Group Info

- Ruihang Lai (ruihangl@cs.cmu.edu)
- Lauren Sands (lsands@cs.cmu.edu)

3 Project Description

Machine learning (ML) applications are now ubiquitous in everyday life. PyTorch [4], as the most popular machine learning framework, adopts the eager execution mode to run models, which follows the execution order of the Python interpreter. Under such eager mode, machine learning frameworks fail to obtain a global picture of how the whole model looks like. As a result, eager-mode frameworks (such as PyTorch) usually miss the computational-graph level acceleration opportunities (e.g., operator fusion) of ML models.

Machine learning compilers [2, 3, 6] aim to optimize and accelerate ML models both globally across the whole computational graph and locally within single tensor operators. These compilers typically provide program abstractions (or intermediate representations) of high-level computational graphs and low-level tensor operators, which are the foundation of the compiler optimizations.

Though ML compilers can bring better performance when running models, describing ML models with these compilers' IR require certain amount of engineering and time. To address this challenge, people try to bridge existing model implementations in PyTorch with ML compilers so that no manual modeling with ML compilers' IR is needed. PyTorch later on introduced its graph-level IR Torch FX [5] and supported generating the an FX graph for a given ML model by symbolic tracing. PyTorch2 [1] further introduces TorchDynamo, which dynamically traces a given ML model with actual runtime input value and generates the traced FX graph.

However, the complexity and dynamism of Python language brings challenges for both symbolic and runtime tracing, which may lead to tracing incompleteness or even failure. MagPy [7] is a new method which overcomes some runtime tracing issues of TorchDynamo by monitoring the execution states of the Python interpreter when tracing. MagPy claims a runtime performance improvement of up to 2.88× by being able to generate more complete graphs.

For now, MagPy only supports generating FX graphs after tracing. Our project is about reimplementing and integrating MagPy with the ML compiler TVM [2]. Though a straightforward way of integration is to translate the generated FX graph to TVM's graph-level IR, we think there might be a

potential of leveraging TVM’s language constructs for better performance or better model coverage. So our plan is to reimplement MagPy and directly generate the graph level IR in TVM.

To evaluate our project, we will use the benchmark dataset in MagPy and measure two metrics. Firstly, we will measure the number of successfully instantiated ML models and check if it can match the numbers reported in the MagPy paper. Secondly, we will measure the runtime performance of these instantiated programs with the compiler optimizations in TVM, and check if the improvement can match the MagPy paper.

100% goal We can reimplement MagPy and match the evaluation results as in the MagPy paper.

75% goal We can reimplement MagPy, but the instantiation success rate is less than what MagPy paper reports. There are a few possible reasons that can lead to this result (for example, TVM lacks some operator support and thus cannot represent a ML model).

125% goal We can be better than what reported in the MagPy paper in either metric. If there is a chance we can leverage TVM for better optimizations, then we may be able to achieve better runtime performance.

4 Logistics

Schedule and milestone

- 10/28–11/03: understand the key insights and techniques in MagPy.
- 11/04–11/20: implement MagPy with TVM (**milestone**).
- 11/21–11/30: evaluate the model instantiation success rate and runtime performance.

Literature search As listed in the references.

Resources needed The basic need is a workstation with an NVIDIA GPU. The MagPy paper uses NVIDIA A100 GPU for evaluation. We will evaluate on A100 as well if we can get access to A100 GPU, or otherwise we will evaluate with NVIDIA RTX 4090 which we already have access to.

Getting started We have quickly went through the MagPy paper and obtained the high-level ideas, while we have not got to the bottom. We have expertise in TVM and for now there is no blocker.

5 Project Web Page

<https://github.com/MasterJH5574/tvm/tree/magpy-15745> (subject to change)

References

- [1] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo

- Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '24, page 929–947, New York, NY, USA, 2024. Association for Computing Machinery. 1
- [2] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. Tvm: An automated end-to-end optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pages 578–594, 2018. 1
- [3] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: Scaling compiler infrastructure for domain specific computation. In 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pages 2–14. IEEE, 2021. 1
- [4] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32, 2019. 1
- [5] James Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. torch.fx: Practical program capture and transformation for deep learning in python. In D. Marculescu, Y. Chi, and C. Wu, editors, Proceedings of Machine Learning and Systems, volume 4, page 638–651, 2022. 1
- [6] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, pages 10–19, 2019. 1
- [7] Chen Zhang, Rongchao Dong, Haojie Wang, Runxin Zhong, Jike Chen, and Jidong Zhai. MAGPY: Compiling eager mode DNN programs by monitoring execution states. In 2024 USENIX Annual Technical Conference (USENIX ATC 24), pages 683–698, Santa Clara, CA, July 2024. USENIX Association. 1